

Unit First :- Python Fundamentals

Features of Python:-

1. Free and Open Source

Python language is freely available at the official website and you can download it from the given download link below click on the **Download Python** keyword. Download Python Since it is open-source, this means that source code is also available to the public. So you can download it, use it as well as share it.

2. Easy to code

Python is a high-level programming language. Python is very easy to learn the language as compared to other languages like C, C#, Javascript, Java, etc. It is very easy to code in the Python language and anybody can learn Python basics in a few hours or days. It is also a developer-friendly language.

3. Easy to Read

As you will see, learning Python is quite simple. As was already established, Python's syntax is really straightforward. The code block is defined by the indentations rather than by semicolons or brackets.

4. Object-Oriented Language

One of the key features of Python is Object-Oriented programming. Python supports object-oriented language and concepts of classes, object encapsulation, etc.

5. GUI Programming Support

Graphical User interfaces can be made using a module such as PyQt5, PyQt4, wxPython, or Tk in Python. PyQt5 is the most popular option for creating graphical apps with Python.

6. High-Level Language

Python is a high-level language. When we write programs in Python, we do not need to remember the system architecture, nor do we need to manage the memory.

7. Large Community Support

Python has gained popularity over the years. Our questions are constantly answered by the enormous StackOverflow community. These websites have already provided answers to many questions about Python, so Python users can consult them as needed.

8. Easy to Debug

Excellent information for mistake tracing. You will be able to quickly identify and correct the majority of your program's issues once you understand how to interpret Python's error traces. Simply by glancing at the code, you can determine what it is designed to perform.

9. Python is a Portable language

Python language is also a portable language. For example, if we have Python code for Windows and if we want to run this code on other platforms such as Linux, Unix, and Mac then we do not need to change it, we can run this code on any platform.

10. Python is an Integrated language

Python is also an Integrated language because we can easily integrate Python with other languages like C, C++, etc

Python Interpreter :-

1. Python Source Code (.py file): You write your program in a .py file.
2. Parser and AST (Syntax Check -> Abstract Syntax Tree): Python checks the code for syntax errors and converts it into an AST (Abstract Syntax Tree), which represents the program structure.
3. Bytecode (.pyc file in __pycache__): The AST is compiled into bytecode, a low-level instruction set. This bytecode may also be saved in the __pycache__ folder for reuse.
4. Python Virtual Machine (Executes Bytecode): The PVM executes the bytecode line by line and translates it into machine instructions.
5. Output (Result on Screen): Finally, the program's result is shown on the screen (for example, via print()).

Example

This program takes two inputs as **a** and **b** and prints sum in the third variable which is **c**. It follows sequential as well as functional execution of programs

```
a = 3
b = 7
c = a + b
print(c)
```

Output

```
10
```

Behind the scenes, here's what happens:

- The interpreter reads each line.
- Converts it into tokens (a, =, 3, +, b, etc.).
- Builds an AST and then bytecode.
- PVM executes the bytecode and displays 10.

Structure of Python program:-

Mostly, python statements are written in such a format that one statement is only written in a single line. The interpreter considers the 'new line character' as the terminator of one instruction. But, writing multiple statements per line is also possible that you can find below.

Examples 1: #

```
print('Welcome to MGM')
```

Output:

```
'Welcome to MGM'
```

```
# Example 2
```

```
x = [1, 2, 3, 4]
```

```
# x[1:3] means that start from the index
```

```
# 1 and go upto the index 2
```

```
print(x[1:3])
```

```
""" In the above mentioned format, the first  
index is included, but the last index is not  
included."""
```

Output:

```
[2, 3]
```

Multiple Statements per Line We can also write multiple statements per line, but it is not a good practice as it reduces the readability of the code.

```
# Example
```

```
a = 10; b = 20; c = b + a
```

```
print(a); print(b); print(c)
```

Output:

```
10
```

```
20
```

```
30
```

Comments in Python

Symbols used for writing comments include Hash (#) or Triple Double Quotation marks("""). Hash is used in writing single line comments that do not span multiple lines

```
# Example 1
```

```
##### This example will print Hello World ##### print('Hello World') # This is a comment
```

Variables:-

Variables act as placeholders for data. They allow us to store and reuse values in our program.

```
# Variable 'x' stores the integer value 10
```

```
x = 5
```

```
# Variable 'name' stores the string "Samantha"
```

```
name = "Samantha"
```

```
print(x)
```

```
print(name)
```

Output

```
5
```

```
Samantha
```

Rules for Naming Variables

To use variables effectively, we must follow Python's naming rules:

- Variable names can only contain letters, digits and underscores (_).
- A variable name cannot start with a digit.
- Variable names are case-sensitive (myVar and myvar are different).
- Avoid using Python keywords (e.g., if, else, for) as variable names.

Basic Casting Functions

- int() - Converts compatible values to an integer.
- float() - Transforms values into floating-point numbers.
- str() - Converts any data type into a string.

```
# Casting variables
```

```
s = "10" # Initially a string
```

```
n = int(s) # Cast string to integer
```

```
cnt = 5
```

```
f = float(cnt) # Cast integer to float
```

```
age = 25
```

```
s2 = str(age) # Cast integer to string
```

```
# Display results
```

```
print(n)
```

```
print(f)
```

```
print(s2)
```

Output

```
10
```

```
5.0
```

```
25
```

Operators:-

In Python programming, Operators in general are used to perform operations on values and variables. These are standard symbols used for logical and arithmetic operations

1)Arithmetic Operators

Python Arithmetic operators are used to perform basic mathematical operations like addition, subtraction, multiplication and division.

```
# Variables
```

```
a = 15
```

```
b = 4
```

```
# Addition
```

```
print("Addition:", a + b)
```

```
# Subtraction
```

```
print("Subtraction:", a - b)
```

```
# Multiplication
```

```
print("Multiplication:", a * b)
```

```
# Division
```

```
print("Division:", a / b)
```

```
# Floor Division
```

```
print("Floor Division:", a // b)
```

```
# Modulus
```

```
print("Modulus:", a % b)
```

```
# Exponentiation
```

```
print("Exponentiation:", a ** b)
```

Output

```
Addition: 19
```

```
Subtraction: 11
```

```
Multiplication: 60
```

```
Division: 3.75
```

```
Floor Division: 3
```

```
Modulus: 3
```

```
Exponentiation: 50625
```

2)Comparison Operators

In Python, Comparison (or Relational) operators compares values. It either returns True or False according to the condition.

```
a=13
```

```
b=33
```

```
print(a > b)
```

```
print(a < b)
```

```
print(a == b)
```

```
print(a != b)
```

```
print(a >= b)
```

```
print(a <= b)
```

Output

```
False
```

```
True
```

```
False
```

```
True
```

```
False
```

```
True
```

3)Logical Operators

Python Logical operators perform Logical AND, Logical OR and Logical NOT operations. It is used to combine conditional statements.

```
a = True
b = False
print(a and b)
print(a or b)
print(not a)
```

Output

```
False
True
False
```

4)Bitwise Operators

Python Bitwise operators act on bits and perform bit-by-bit operations. These are used to operate on binary numbers.

```
a = 10
b = 4
print(a & b)
print(a | b)
print(~a)
print(a ^ b)
print(a >> 2)
print(a << 2)
```

Output

```
0
14
-11
```

14

2

40

5) Assignment Operators

Python Assignment operators are used to assign values to the variables. This operator is used to assign the value of the right side of the expression to the left side operand.

```
a = 10
```

```
b = a
```

```
print(b)
```

```
b += a
```

```
print(b)
```

```
b -= a
```

```
print(b)
```

```
b *= a
```

```
print(b)
```

```
b <<= a
```

```
print(b)
```

Output

10

20

10

100

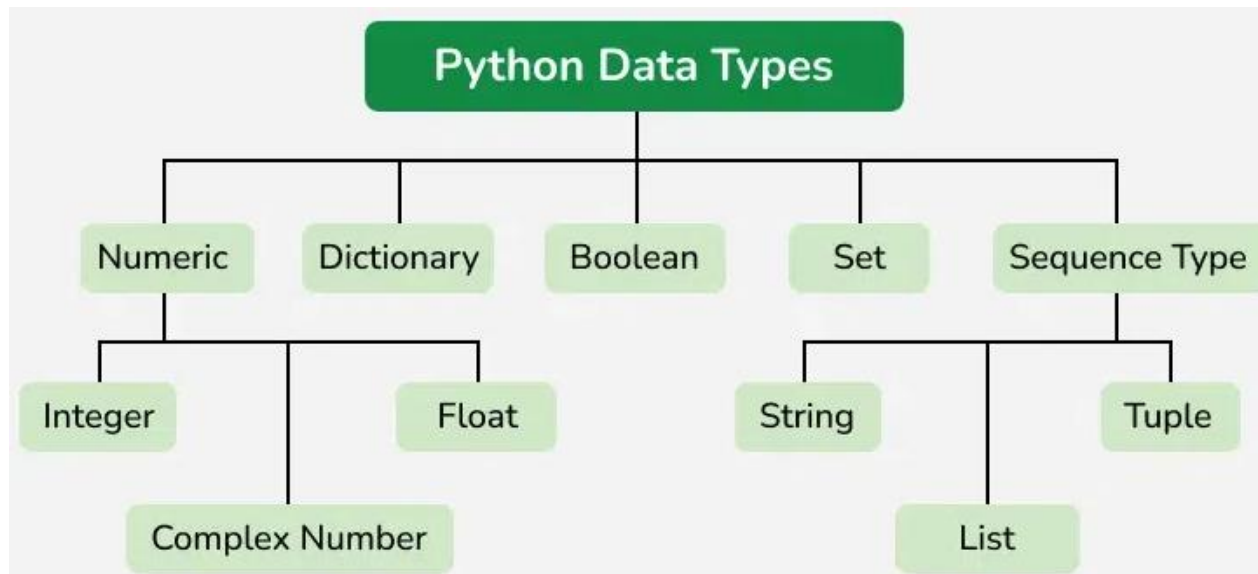
102400

Data types :-

Data types in Python are a way to classify data items. They represent the kind of value, which determines what operations can be performed on that data. Since everything is an object in Python programming, Python data types are classes and variables are instances (objects) of these classes.

The following are standard or built-in data types in Python:

- Numeric: int, float, complex
- Sequence Type: string, list, tuple
- Mapping Type: dict
- Boolean: bool
- Set Type: set, frozenset
- Binary Types: bytes, bytearray, memoryview



1. Numeric Data Types

Python numbers represent data that has a numeric value. A numeric value can be an integer, a floating number or even a complex number. These values are defined as int, float and complex classes.

- **Integers:** value is represented by int class. It contains positive or negative whole numbers (without fractions or decimals). There is no limit to how long an integer value can be.
- **Float:** value is represented by float class. It is a real number with a floating-point representation. It is specified by a decimal point. Optionally, character e or E followed by a positive or negative integer may be appended to specify scientific notation.
- **Complex Numbers:** It is represented by a complex class. It is specified as (real part) + (imaginary part)j. For example - 2+3j

a = 5

```
print(type(a))
```

b = 5.0

```
print(type(b))
```

c = 2 + 4j

```
print(type(c))
```

Output

```
<class 'int'>  
<class 'float'>  
<class 'complex'>
```

2. Sequence Data Types

A sequence is an ordered collection of items, which can be of similar or different data types.

String Data Type:-

Python Strings are arrays of bytes representing Unicode characters. In Python, there is no character data type, a character is a string of length one. It is represented by str class.

```
s = 'Welcome to the Geeks World'
```

```
print(s)
```

```
# check data type
```

```
print(type(s))
```

```
# access string with index
```

```
print(s[1])
```

```
print(s[2])
```

```
print(s[-1])
```

Output

```
Welcome to the Geeks World  
<class 'str'>  
e  
l  
d
```

List Data Type

Lists are similar to arrays found in other languages. They are an ordered and mutable collection of items

```
# Empty list
```

```
a = []  
  
# list with int values  
  
a = [1, 2, 3]  
  
print(a)  
  
# list with mixed values int and String  
  
b = ["Geeks", "For", "Geeks", 4, 5]  
  
print(b)
```

Output

```
[1, 2, 3]  
['Geeks', 'For', 'Geeks', 4, 5]
```

Tuple Data Type

Tuple is an ordered collection of Python objects. The only difference between a tuple and a list is that tuples are immutable. Tuples cannot be modified after it is created.

```
# initiate empty tuple  
  
tup1 = ()  
  
tup2 = ('Geeks', 'For')  
  
print("\nTuple with the use of String: ", tup2)
```

Output

```
Tuple with the use of String: ('Geeks', 'For')
```

3. Boolean Data Type in Python

Data type is one of the two built-in values, True or False. Boolean objects that are equal to True are truthy (true) and those equal to False are falsy (false). However non-Boolean objects can be evaluated in a Boolean context as well and determined to be true or false. It is denoted by class bool.

Example: First two lines will print type of the boolean values True and False, which is <class 'bool'>. Third line will cause an error, because true is not a valid keyword. Python is case-sensitive, which means it distinguishes between uppercase and lowercase letters.

```
print(type(True))  
  
print(type(False))
```

```
print(type(True))
```

Output:

```
<class 'bool'>
```

```
<class 'bool'>
```

4. Set Data Type in Python

In Python Data Types, Set is an unordered collection of data types that is iterable, mutable, and has no duplicate elements.

```
# initializing empty set
```

```
s1 = set()
```

```
s1 = set("GeeksForGeeks")
```

```
print("Set with the use of String: ", s1)
```

```
s2 = set(["Geeks", "For", "Geeks"])
```

```
print("Set with the use of List: ", s2)
```

Output

```
Set with the use of String: {'s', 'o', 'F', 'G', 'e', 'k', 'r'}
```

```
Set with the use of List: {'Geeks', 'For'}
```

5. Dictionary Data Type

A dictionary in Python is a collection of data values, used to store data values like a map, unlike other Python Data Types, a Dictionary holds a key: value pair. Key-value is provided in dictionary to make it more optimized. Each key-value pair in a Dictionary is separated by a colon :, whereas each key is separated by a 'comma'.

```
# initialize empty dictionary
```

```
d = {}
```

```
d = {1: 'Geeks', 2: 'For', 3: 'Geeks'}
```

```
print(d)
```

```
# creating dictionary using dict() constructor
```

```
d1 = dict({1: 'Geeks', 2: 'For', 3: 'Geeks'})  
  
print(d1)
```

Output

```
{1: 'Geeks', 2: 'For', 3: 'Geeks'}  
{1: 'Geeks', 2: 'For', 3: 'Geeks'}
```

I/O statements :-

Python provides functions to **take input from the user** (Input) and **display output on the screen** (Output).

1 Output Statements – `print()`

The `print()` function is used to display information.

Example:

```
# Simple output  
print("Hello, Python!")  
  
# Printing multiple values  
x = 10  
y = 20  
print("x =", x, "y =", y)  
  
# Using sep and end parameters  
print("Apple", "Banana", "Cherry", sep=", ") # Custom separator  
print("Line without newline", end=" ") # Prevents new line  
print("Next text")
```

✓Output:

```
Hello, Python!  
x = 10 y = 20  
Apple, Banana, Cherry  
Line without newline Next text
```

2 Input Statements – `input()`

The `input()` function allows the user to enter data from the keyboard.

△ By default, input is **always read as a string**.

Example:

```
# Taking input as string
name = input("Enter your name: ")
print("Hello,", name)

# Taking integer input
age = int(input("Enter your age: "))    # type casting to int
print("You are", age, "years old")

# Taking float input
marks = float(input("Enter your marks: "))
print("You scored:", marks)
```

3 Formatted Output

Python has multiple ways to format output neatly.

(a) f-strings (Python 3.6+)

```
name = "Manisha"
age = 21
print(f"My name is {name} and I am {age} years old.")
```

(b) str.format()

```
print("My name is {} and I am {} years old.".format(name, age))
```

(c) Old % formatting

```
print("My name is %s and I am %d years old." % (name, age))
```

✓Output (for all three):

My name is Manisha and I am 21 years old.

4 File Input/Output (Basic)

Python also allows reading and writing to files.

Example:

```
# Writing to a file
with open("sample.txt", "w") as f:
    f.write("Hello, this is Python file I/O!")

# Reading from a file
with open("sample.txt", "r") as f:
    content = f.read()
```

```
print("File content:", content)
```

Control structures :-

Control structures determine **the flow of execution** in a Python program. They are mainly of three types:

1. Conditional Statements (Decision Making)

if statement

Executes a block of code if the condition is true.

```
x = 10
if x > 5:
    print("x is greater than 5")
```

if...else

Executes one block if the condition is true, otherwise another.

```
x = 3
if x % 2 == 0:
    print("Even number")
else:
    print("Odd number")
```

if...elif...else

Checks multiple conditions.

```
marks = 75
if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
elif marks >= 50:
    print("Grade C")
else:
    print("Fail")
```

2. Looping Statements (Iteration)

for loop

Used to iterate over a sequence (list, string, tuple, range, etc.).

```
for i in range(1, 6):  
    print(i)    # prints numbers 1 to 5
```

while loop

Repeats a block as long as the condition is true.

```
n = 1  
while n <= 5:  
    print(n)  
    n += 1
```

3. Jump Statements

break

Exits the loop immediately.

```
for i in range(1, 10):  
    if i == 5:  
        break  
    print(i)    # stops at 4
```

continue

Skips the current iteration and continues.

```
for i in range(1, 6):  
    if i == 3:  
        continue  
    print(i)    # skips 3
```

pass

Does nothing (used as a placeholder).


```
for i in range(5):  
    pass    # no operation
```

4. Nested Control Structures

Control structures can be used inside one another.

```
for i in range(1, 4):  
    if i % 2 == 0:  
        print(i, "is Even")  
    else:  
        print(i, "is Odd")
```

Decision and loop python

1. Decision (Conditional) Statements

These allow Python to **make decisions** and execute different code blocks depending on conditions.

(a) if statement

```
x = 10  
if x > 5:  
    print("x is greater than 5")
```

(b) if...else

```
x = 3  
if x % 2 == 0:  
    print("Even number")  
else:  
    print("Odd number")
```

(c) if...elif...else

```
marks = 85  
if marks >= 90:  
    print("Grade A")  
elif marks >= 75:  
    print("Grade B")  
elif marks >= 50:  
    print("Grade C")
```

```
else:  
    print("Fail")
```

2. Looping Statements

These allow Python to **repeat a block of code** multiple times.

(a) for loop

Iterates over a sequence (list, string, tuple, range, etc.).

```
for i in range(1, 6):  
    print(i)    # prints 1 to 5
```

(b) while loop

Repeats as long as the condition is true.

```
n = 1  
while n <= 5:  
    print(n)  
    n += 1
```

3. Loop Control Statements

break → Exit loop immediately

```
for i in range(1, 10):  
    if i == 5:  
        break  
    print(i)
```

continue → Skip current iteration

```
for i in range(1, 6):  
    if i == 3:  
        continue  
    print(i)
```

pass → Do nothing (placeholder)

```
for i in range(5):
```

```
pass # No operation
```

String and string Operations

In Python, a string is a sequence of characters enclosed in quotes. It can include letters, numbers, symbols or spaces. Since Python has no separate character type, even a single character is treated as a string with length one. Strings are widely used for text handling and manipulation.

Creating a String

Strings can be created using either **single** ('...') or **double** ("...") quotes. Both behave the same.

Example: Creating two equivalent strings one with single and other with double quotes.

```
s1 = 'GfG' # single quote
s2 = "GfG" # double quote

print(s1)
print(s2)
```

Output

```
GfG
```

```
GfG
```

Multi-line Strings

Use triple quotes ('''...''') or ("""...""") for strings that span multiple lines. Newlines are preserved.

Example: Define and print multi-line strings using both styles.

```
s = '''I am Learning
```

```
Python String on GeeksforGeeks''''
```

```
print(s)
```

```
s = """I'm a
```

```
Geek"""
```

```
print(s)
```

Output

```
I am Learning
```

```
Python String on GeeksforGeeks
```

```
I'm a  
Geek
```

Accessing characters in String

Strings are indexed sequences. Positive indices start at **0** from the **left**; negative indices start at **-1** from the **right** as represented in below image:

Example 1: Access specific characters through										Competitive Programming -		
0	1	2	3	4	5	6	7	8	9	10	11	12
G	E	E	K	S	F	O	R	G	E	E	K	S
-13	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

Example 1: Access specific characters through positive indexing.

```
s = "GeeksforGeeks"  
  
print(s[0]) # first character  
  
print(s[4]) # 5th character
```

Output

```
G  
  
S
```

Example 2: Read characters from the end using [negative indices](#).

```
s = "GeeksforGeeks"  
  
print(s[-10]) # 3rd character  
  
print(s[-5]) # 5th character from end
```

Output

```
k  
  
G
```

String Slicing

Slicing is a way to extract a portion of a string by specifying the **start** and **end** indexes. The syntax for slicing is **string[start:end]**, where **start** starting index and **end** is stopping index (excluded).

Example: In this example we are slicing through range and reversing a string.

```
s = "GeeksforGeeks"
```

```
print(s[1:4]) # characters from index 1 to 3
```

```
print(s[:3]) # from start to index 2
```

```
print(s[3:]) # from index 3 to end
```

```
print(s[::-1]) # reverse string
```

Output

```
eek
```

```
Gee
```

```
ksforGeeks
```

```
skeeGrofskeeG
```

String Iteration

Strings are iterable; you can loop through characters one by one.

Example: Here, it print each character on its own line.

```
s = "Python"
```

```
for char in s:
```

```
    print(char)
```

Output

```
P
```

```
y
```

```
t
```

```
h
```

```
o
```

```
n
```

Deleting a String

In Python, it is not possible to delete individual characters from a string since strings are immutable. However, we can delete an entire string variable using the del keyword.

Example: Here, we are using del keyword to delete a string.

```
s = "GfG"
```

```
del s
```

Updating a String

As strings are immutable, “updates” create new strings using slicing or methods such as replace().

Example: This code fix the first letter and replace a word.

```
s = "hello geeks"
```

```
s1 = "H" + s[1:]          # update first character
```

```
s2 = s.replace("geeks", "GeeksforGeeks") # replace word
```

```
print(s1)
```

```
print(s2)
```

Output

```
Hello geeks
```

```
hello GeeksforGeeks
```

Common String Methods

Python provides various built-in methods to manipulate strings. Below are some of the most useful methods:

1. **len():** The len() function returns the total number of characters in a string (including spaces and punctuation).

Example:

```
s = "GeeksforGeeks"
```

```
print(len(s))
```

Output

```
13
```

2. **upper() and lower():** upper() method converts all characters to uppercase whereas, lower() method converts all characters to lowercase.

Example:

```
s = "Hello World"
```

```
print(s.upper())
```

```
print(s.lower())
```

Output

```
HELLO WORLD
```

```
hello world
```

3. **strip() and replace():** strip() removes leading and trailing whitespace from the string and replace() replaces all occurrences of a specified substring with another.

Example:

```
s = " Gfg "
```

```
print(s.strip())
```

```
s = "Python is fun"
```

```
print(s.replace("fun", "awesome"))
```

Output

```
Gfg
```

```
Python is awesome
```

Concatenating and Repeating Strings:-

We can concatenate strings using + operator and repeat them using * operator.

1. Strings can be combined by using + **operator**.

Example: Join two words with a space.

```
s1 = "Hello"
```

```
s2 = "World"
```

```
print(s1 + " " + s2)
```

Output

```
Hello World
```

2. We can repeat a string multiple times using * operator.

Example: Repeat a greeting three times.

```
s = "Hello "
```

```
print(s * 3)
```

Output

```
Hello Hello Hello
```

SET

A Set in Python is used to store a collection of items with the following properties.

No duplicate elements. If try to insert the same item again, it overwrites previous one.

An unordered collection. When we access all items, they are accessed without any specific order and we cannot access items using indexes as we do in lists.

Internally use hashing that makes set efficient for search, insert and delete operations. It gives a major advantage over a list for problems with these operations.

Mutable, meaning we can add or remove elements after their creation, the individual elements within the set cannot be changed directly.

Example of Python Sets :

```
s = {10, 50, 20}
```

```
print(s)
print(type(s))
```

Output

```
{10, 50, 20}
<class 'set'>
```

Type Casting with Python Set method

The Python set() method is used for type casting

typecasting list to set

```
s = set(["a", "b", "c"])
```

```
print(s)
```

Adding element to the set

```
s.add("d")
```

```
print(s)
```

Output

```
{'c', 'b', 'a'}
{'d', 'c', 'b', 'a'}
```

Check unique and Immutable with Python Set

Python sets cannot have duplicate values. While you cannot modify the individual elements directly, you can still add or remove elements from the set.

Python program to demonstrate that

a set cannot have duplicate values

and we cannot change its items

a set cannot have duplicate values

```
s = {"Geeks", "for", "Geeks"}
```

```
print(s)
```

values of a set cannot be changed


```
s[1] = "Hello"
```

```
print(s)
```

Output:

The first code explains that the set cannot have a duplicate value. Every item in it is a unique value.

The second code generates an error because we cannot assign or change a value once the set is created. We can only add or delete items in the set.

```
{ 'Geeks', 'for' }
```

```
TypeError: 'set' object does not support item assignment
```

Python Dictionary

Python dictionary is a data structure that stores the value in key: value pairs. Values in a dictionary can be of any data type and can be duplicated, whereas keys can't be repeated and must be immutable.

- Keys are case sensitive which means same name but different cases of Key will be treated distinctly.
- Keys must be immutable which means keys can be strings, numbers or tuples but not lists.
- Duplicate keys are not allowed and any duplicate key will overwrite the previous value.
- Internally uses hashing. Hence, operations like search, insert, delete can be performed in Constant Time.

How to Create a Dictionary

Dictionary can be created by placing a sequence of elements within curly {} braces, separated by a 'comma'.

```
d1 = {1: 'Geeks', 2: 'For', 3: 'Geeks'}
```

```
print(d1)
```

```
# create dictionary using dict() constructor
```

```
d2 = dict(a = "Geeks", b = "for", c = "Geeks")
```

```
print(d2)
```

Output

```
{1: 'Geeks', 2: 'For', 3: 'Geeks'}
```

```
{'a': 'Geeks', 'b': 'for', 'c': 'Geeks'}
```

Accessing Dictionary Items

We can access a value from a dictionary by using the **key** within square brackets or [get\(\)](#) method.

```
d = { "name": "Prajjwal", 1: "Python", (1, 2): [1,2,4] }
```

```
# Access using key
```

```
print(d["name"])
```

```
# Access using get()
```

```
print(d.get("name"))
```

Output

```
Prajjwal
```

```
Prajjwal
```

Adding and Updating Dictionary Items

We can add new key-value pairs or update existing keys by using assignment.

```
d = {1: 'Geeks', 2: 'For', 3: 'Geeks'}
```

```
# Adding a new key-value pair
```

```
d["age"] = 22
```

```
# Updating an existing value
```

```
d[1] = "Python dict"
```

```
print(d)
```

Output

```
{1: 'Python dict', 2: 'For', 3: 'Geeks', 'age': 22}
```

Removing Dictionary Items

We can remove items from dictionary using the following methods:

- **del:** Removes an item by key.
- **pop():** Removes an item by key and returns its value.
- **clear():** Empties the dictionary.
- **popitem():** Removes and returns the last key-value pair.

```
d = {1: 'Geeks', 2: 'For', 3: 'Geeks', 'age': 22}
```

```
# Using del to remove an item

del d["age"]

print(d)

# Using pop() to remove an item and return the value

val = d.pop(1)

print(val)

# Using popitem to removes and returns

# the last key-value pair.

key, val = d.popitem()

print(f"Key: {key}, Value: {val}")

# Clear all items from the dictionary

d.clear()

print(d)
```

Output

```
{1: 'Geeks', 2: 'For', 3: 'Geeks'}
Geeks
Key: 3, Value: Geeks
{}
```

Iterating Through a Dictionary

We can iterate over **keys** [using keys() method] , **values** [using values() method] or both [using item() method] with a for loop.

```
d = {1: 'Geeks', 2: 'For', 'age':22}
```

```
# Iterate over keys
```

```
for key in d:
```

```
    print(key)
```

```
# Iterate over values
for value in d.values():
    print(value)

# Iterate over key-value pairs
for key, value in d.items():
    print(f"{key}: {value}")
```

Output

```
1
2
age
Geeks
For
22
1: Geeks
2: For
age: 22
```

Python Lists

In Python, a list is a built-in data structure that can hold an ordered collection of items. Unlike arrays in some languages, Python lists are very flexible:

- Can contain duplicate items
- **Mutable:** items can be modified, replaced, or removed
- **Ordered:** elements maintain the order in which they were added
- **Index-based:** items are accessed using their position (starting from 0)
- Can store mixed data types (integers, strings, booleans, even other lists)

Creating a List

Lists can be created in several ways, such as using square brackets, the list() constructor or by repeating elements. Let's look at each method one by one with example:

1. Using Square Brackets

We use square brackets [] to create a list directly.

Example: Creating lists with integers, strings and mixed data types.

```
a = [1, 2, 3, 4, 5] # List of integers
```

```
b = ['apple', 'banana', 'cherry'] # List of strings
```

```
c = [1, 'hello', 3.14, True] # Mixed data types
```

```
print(a)
```

```
print(b)
```

```
print(c)
```

Output

```
[1, 2, 3, 4, 5]
```

```
['apple', 'banana', 'cherry']
```

```
[1, 'hello', 3.14, True]
```

2. Using list() Constructor

We can also create a list by passing an iterable (like a tuple, string or another list) to the list() function.

Example: Creating a list from tuple and string.

```
a = list((1, 2, 3, 'apple', 4.5))
```

```
print(a)
```

```
b = list("GFG")
```

```
print(b)
```

Output

```
[1, 2, 3, 'apple', 4.5]
```

```
['G', 'F', 'G']
```

3. Creating List with Repeated Elements

We can use the multiplication operator * to create a list with repeated items.

Example: Creating lists with repeated values.

```
a = [2] * 5
```

```
b = [0] * 7
```

```
print(a)
```

```
print(b)
```

Output

```
[2, 2, 2, 2, 2]
```

```
[0, 0, 0, 0, 0, 0, 0]
```

Accessing List Elements

Elements in a list are accessed using indexing. Python indexes start at 0, so `a[0]` gives the first element. Negative indexes allow access from the end (e.g., `-1` gives the last element).

Example: Accessing first, last and slicing elements.

```
a = [10, 20, 30, 40, 50]
```

```
print(a[0])
```

```
print(a[-1])
```

```
print(a[1:4]) # elements from index 1 to 3
```

Output

```
10
50
[20, 30, 40]
```

Adding Elements into List

We can add elements to a list using the following methods:

append(): Adds an element at the end of the list.

extend(): Adds multiple elements to the end of the list.

insert(): Adds an element at a specific position.

clear(): removes all items.

Example: Demonstrating [append](#), [insert](#), [clear](#) and [extend](#).

```
a = []
```

```
a.append(10)
```

```
print("After append(10):", a)
```

```
a.insert(0, 5)
```

```
print("After insert(0, 5):", a)
```

```
a.extend([15, 20, 25])
```

```
print("After extend([15, 20, 25]):", a)
```

```
a.clear()
```

```
print("After clear():", a)
```

Output

```
After append(10): [10]
After insert(0, 5): [5, 10]
```

```
After extend([15, 20, 25]): [5, 10, 15, 20, 25]
```

```
After clear(): []
```

Updating Elements into List

Since lists are mutable, we can update elements by accessing them via their index.

Example: Updating the second element.

```
a = [10, 20, 30, 40, 50]
```

```
a[1] = 25
```

```
print(a)
```

Output

```
[10, 25, 30, 40, 50]
```

Removing Elements from List

We can remove elements from a list using:

remove(): Removes the first occurrence of an element.

pop(): Removes the element at a specific index or the last element if no index is specified.

del statement: Deletes an element at a specified index.

Example: Demonstrating remove, pop and del

```
a = [10, 20, 30, 40, 50]
```

```
a.remove(30)
```

```
print("After remove(30):", a)
```

```
popped_val = a.pop(1)
```

```
print("Popped element:", popped_val)
```

```
print("After pop(1):", a)
```

```
del a[0]
```

```
print("After del a[0]:", a)
```

Output

```
After remove(30): [10, 20, 40, 50]
```

```
Popped element: 20
```

```
After pop(1): [10, 40, 50]
```

```
After del a[0]: [40, 50]
```

Python Tuples

A tuple in Python is an immutable ordered collection of elements.

Tuples are similar to lists, but unlike lists, they cannot be changed after their creation (i.e., they are immutable).

Tuples can hold elements of different data types.

The main characteristics of tuples are being ordered, heterogeneous and immutable.

Creating a Tuple

A tuple is created by placing all the items inside parentheses (), separated by commas. A tuple can have any number of items and they can be of different data types.

```
tup = ()
```

```
print(tup)
```

```
# Using String
```

```
tup = ('Geeks', 'For')
```

```
print(tup)
```

```
# Using List
```

```
li = [1, 2, 4, 5, 6]
```

```
print(tuple(li))
```

```
# Using Built-in Function
```

```
tup = tuple('Geeks')
```

```
print(tup)
```

Output

```
()  
( 'Geeks', 'For' )  
(1, 2, 4, 5, 6)  
( 'G', 'e', 'e', 'k', 's' )
```

Let's understand tuple in detail:

Creating a Tuple with Mixed Datatypes.

Tuples can contain elements of various data types, including other tuples, [lists](#), [dictionaries](#) and even [functions](#).

```
tup = (5, 'Welcome', 7, 'Geeks')
```

```
print(tup)
```



```
# Creating a Tuple with nested tuples

tup1 = (0, 1, 2, 3)

tup2 = ('python', 'geek')

tup3 = (tup1, tup2)

print(tup3)

# Creating a Tuple with repetition

tup1 = ('Geeks',) * 3

print(tup1)

# Creating a Tuple with the use of loop

tup = ('Geeks')

n = 5

for i in range(int(n)):

    tup = (tup,)

    print(tup)
```

Output

```
(5, 'Welcome', 7, 'Geeks')
((0, 1, 2, 3), ('python', 'geek'))
('Geeks', 'Geeks', 'Geeks')
('Geeks',)
(('Geeks',),)
((( 'Geeks',),),)
(((( 'Geeks',),),),)
((((('Geeks',),),),),)
```

Functions and Arguments:-

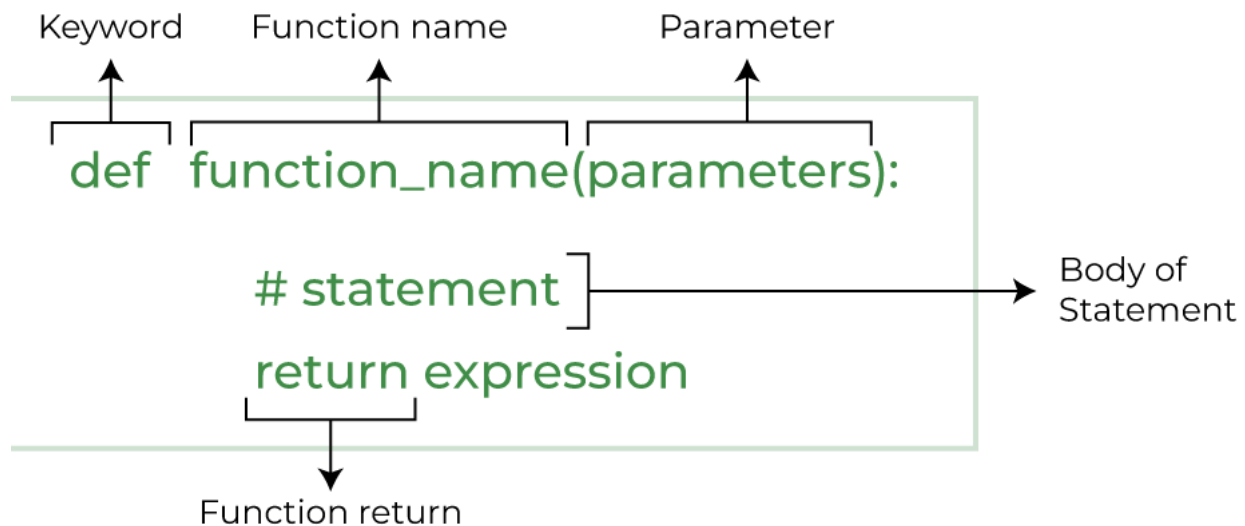
Python Functions

Python Functions are a block of statements that does a specific task. The idea is to put some commonly or repeatedly done task together and make a

function so that instead of writing the same code again and again for different inputs, we can do the function calls to reuse code contained in it over and over again.

Function Declaration

The syntax to declare a function is:



Defining a Function

We can define a function in Python, using the **def keyword**. We can add any type of functionalities and properties to it as we require.

The def keyword stands for define. It is used to create a **user-defined function**. It marks the beginning of a function block and allows you to group a set of statements so they can be reused when the function is called.

Syntax:

```
def function_name(parameters):  
# function body
```

Explanation:

- **def:** Starts the function definition.
- **function_name:** Name of the function.
- **parameters:** Inputs passed to the function (inside ()), optional.
- **Indented code:** The function body that runs when called.

Example: Let's understand this with a simple example. Here, we define a function using `def` that prints a welcome message when called.

```
def fun():
```

```
print("Welcome to GFG")
```

Calling a Function

After creating a function in Python we can call it by using the name of the functions followed by parenthesis containing parameters of that particular function. Below is the example for calling def function Python

```
def fun():
```

```
    print("Welcome to GFG")
```

```
fun() # Driver code to call a function
```

Output

```
Welcome to GFG
```

Function Arguments

Arguments are the values passed inside the parenthesis of the function. A function can have any number of arguments separated by a comma.

Syntax:

```
def function_name(parameters):  
    """Docstring"""  
    # body of the function  
    return expression
```

Let's understand this with an example, we will create a simple function in Python to check whether the number passed as an argument to the function is even or odd.

```
def evenOdd(x):
```

```
    if (x % 2 == 0):
```

```
        return "Even"
```

```
    else:
```

```
        return "Odd"
```

```
print(evenOdd(16))
```

```
print(evenOdd(7))
```

Output

```
Even
```

Odd

Types of Function Arguments

Python supports various types of arguments that can be passed at the time of the function call. In Python, we have the following function argument types in Python, Let's explore them one by one.

1. Default Arguments

A [default argument](#) is a parameter that assumes a default value if a value is not provided in the function call for that argument. The following example illustrates Default arguments to write

```
def myFun(x, y=50):
```

```
    print("x: ", x)
```

```
    print("y: ", y)
```

```
myFun(10)
```

Output

```
x:  10
```

```
y:  50
```

2. Keyword Arguments

In keyword arguments, values are passed by explicitly specifying the parameter names, so the order doesn't matter.

```
def student(fname, lname):
```

```
    print(fname, lname)
```

```
student(fname='Geeks', lname='Practice')
```

```
student(lname='Practice', fname='Geeks')
```

Output

```
Geeks Practice
```

```
Geeks Practice
```